

CCOP v1.0 — 프로젝트 종합 설명서

작성일: 2026-07-14 · **대상 독자:** 이 프로젝트를 처음 접하는 개발자·평가자·인수인계 대상자 **목적:** 이 문서 하나로 ① 어떤 프로젝트인지 ② 어떤 기능이 있는지 ③ 어떻게 만들어졌는지 파악할 수 있게 한다. **수치 기준:** 2026-07 실측 (코드·문서·벤치마크 결과 파일 기준)

목차

- 한눈에 보기
- 무엇을 왜 만들었나 (문제 정의)
- 핵심 개념 4가지
- 기능 전체 (사용자 관점)
- 시스템 아키텍처
- 코드 구조 (디렉토리 맵)
- AI 파이프라인 — 어떻게 만들어졌나
- 온톨로지 — 도메인 데이터 모델
- 데이터 적재 파이프라인
- 보안 설계
- 배포와 운영 (3개 환경)
- 품질 관리 (테스트·CI·벤치마크)
- 개발 타임라인
- 빠른 시작
- 더 읽을 문서 지도
- 용어집

1. 한눈에 보기

CCOP(Cybercrime Investigation Graph Platform)는 사이버범죄 수사관을 위한 그래프 분석 플랫폼이다. 사건·인물·계좌·전화·IP가 얽힌 관계 데이터를 그래프 DB에 적재하고, 수사관이 **한국어로 질문하면 그래프 쿼리(Cypher)로 자동 변환**해 시각화된 답을 보여준다. 핵심 차별점은 이 변환을 **외부 API 없이 사내 GPU에서 도는 자체 파인튜닝 모델(sLLM)**이 수행한다는 것 — 수사 데이터를 외부로 보낼 수 없는 망분리 환경을 전제로 설계됐다.

항목	내용
기간	2026.01 ~ 진행 중 (1인 개발: 기획·연구·개발·운영)
스택 요약	Python/Flask · AgensGraph(PostgreSQL 그래프 확장) · LangGraph · vLLM · Qwen2.5-7B(QLoRA 파인튜닝) · Cytoscape.js · Docker
규모	백엔드 32파일 ~17,500 LOC · REST 엔드포인트 86개 · 서비스 모듈 20개 · 프론트 템플릿 ~9,800 LOC
AI 성과	자연어→Cypher 정확도 86.6% (자체 232문항 벤치마크, 운영 임계 85% 초과) — 파인튜닝 모델 v42 + 규칙 Router
데이터	자체 구축 SFT 31,226샘플 · 운영 그래프 9개 (최대 689만 노드/1,094만 엣지 OSINT 그래프)
품질	테스트 함수 190여 개 (CI 오프라인 게이트 171개) · 기술 문서 84건
운영	클라우드 VM에서 e2e 1.4초로 가동 중 · 폐쇄망(air-gap) 설치 체계 구축

2. 무엇을 왜 만들었나 (문제 정의)

사이버범죄(보이스피싱·몸캠피싱·투자사기 등) 수사의 본질은 **관계 추적**이다: 피해자 계좌에서 출발해 이체 체인, 대포통장, 공유된 전화번호·IP, 배후 조직으로 이어지는 연결을 찾는 일. 관계형 DB와 엑셀로는 이 "몇 다리 건너 연결"(multi-hop) 질문에 답하기 어렵고, 그래프 DB가 정확한 도구다. 그런데 세 가지 장벽이 있었다:

1. **수사관은 그래프 쿼리 언어(Cypher)를 모른다.** → 자연어로 질문하면 Cypher로 변환하는 **Text2Cypher** 계층이 필요.
2. **수사 데이터는 외부 API로 보낼 수 없다** (망분리·폐쇄망 규정). → OpenAI 의존이 아닌 **온프레미스 sLLM**이 필요. 범용 7B 모델은 이 도메인 스키마를 모르므로 **직접 파인튜닝**해야 한다.
3. **기관마다 데이터 포맷이 제각각**이고(KICS, 금융권, 통신사, OSINT), 도메인 개념(사건·진정서·접근매체·중계기)이 일반 스키마에 없다. → 형사사법 표준(KICS)에 정렬한 **자체 온톨로지**와 **적재 파이프라인**이 필요.

CCOP는 이 세 가지를 모두 자체 구축한 결과물이다: **온톨로지(스키마) + 파인튜닝 모델(변환기) + 플랫폼(적재·탐색·시각화·분석)**.

3. 핵심 개념 4가지

처음 보는 사람이 이 네 개념만 잡으면 나머지가 읽힌다.

① **Text2Cypher** — "익명 피의자가 보유한 계좌 보여줘" 같은 한국어 질문을 `MATCH (p:vt_psn {is_anonymous: true})-[:has_account]->(a:vt_bacnt) RETURN ...` 같은 그래프 쿼리로 바꾸는 것. Text2SQL의 그래프 DB 버전이며, 이 프로젝트의 AI 핵심 과제다.

② **온톨로지 (POLE 6계층, 25노드/53엣지)** — "이 도메인에 어떤 개체와 관계가 존재하는가"의 표준 정의. 수사 데이터 모델링의 국제 관행인 POLE(Person·Object·Location·Event)을 확장해 Source·Case를 더한 6계층으로 설계했다. 학습 데이터 생성, ETL, 시각화, 쿼리 검증이 전부 이 단일 정의(SSOT)를 바라본다. — §8

③ **sLLM (small/specialized LLM)** — GPT-4o 같은 외부 대형 모델 대신, 사내 GPU에서 도는 7B급 모델(Qwen2.5-7B)을 이 도메인 전용으로 파인튜닝한 것. 자체 벤치마크에서 GPT-4o(95.4%)에는 못 미치지만 운영 임계(85%)를 넘는 86.6%를 달성했고, 데이터가 외부로 나가지 않는다. — §7

④ **AgensGraph** — PostgreSQL 기반 상용 그래프 DB. 하나의 인스턴스에서 관계형 테이블(RDB)과 그래프(GDB)를 함께 다룰 수 있어(One-Instance Multi-Model), 원천 데이터(RDB) ↔ 분석 그래프(GDB) 이중 구조를 단일 DB로 운영한다.

4. 기능 전체 (사용자 관점)

4.1 대표 사용 시나리오 (한 사건의 흐름)

- ① 데이터 반입: 금융/통신 CSV 업로드 → AI가 컬럼→온톨로지 매핑 제안 → 그래프 적재
- ② 자연어 질의: "부산 보이스피싱 사건의 피의자 보여줘" → sLLM이 Cypher 생성·실행 → 그래프 표시
- ③ 탐색: 노드 우클릭 → 1-hop/N-depth 확장, 두 노드 간 경로 분석, 허브(연결 집중점) 탐지
- ④ 분석: 사건 서브그래프를 8종 사기 패턴과 매칭 → "자금세탁체인 87% 일치" + 부족한 증거 목록
- ⑤ 법률 참고: "인출책 처벌 수위는?" → 법률 RAG가 조문·판례 근거와 함께 답변
- ⑥ 공유: 외부 협력기관이 read-only API로 조회 / 서브그래프 저장·내보내기

4.2 기능 카탈로그

영역	기능	구현 위치
자연어 질의	한국어→Cypher 변환(멀티턴 세션, 엔티티 컨텍스트 유지), 실패 시 자동 재시도(reflection), 잡담/위험질의 사전 차단	langgraph_agent.py , ai_service.py , POST /api/query/ai
그래프 탐색	키워드 노드 검색, 1-hop/정밀 N-depth 확장(hairball 방지), 최단 경로, 서브그래프 저장	graph_service.py , /api/search·expand·path
시각화	Cytoscape.js 다크 워크벤치, 노드 타입별 색·아이콘 인코딩(디자인 토큰 SoT), 우클릭 컨텍스트 메뉴, 자동 레이아웃(dagre), 타임라인·스칼라 패널	templates/index.html (~7,000 LOC), static/css/tokens.css
ETL(적재)	CSV 업로드 → LLM 컬럼 매핑 제안 → 표준화(은행·통신사·해시 코드) → 배치 적재 + 메타(출처·신뢰등급) 자동 주입	etl_service.py , schema_mapper.py
RDB→그래프 변환	관계형 원천 49개 테이블을 POLE 그래프로 일괄 변환(증거등급 주입, 역추적 키 유지)	rdb_to_graph_service.py (2,148 LOC)
모델러	비주얼 ETL/자유 그래프 설계 도구 — 사용자가 노드·엣지 구조를 직접 설계하고 Cypher 생성·실행	templates/modeler.html , /api/modeler/*
패턴 분석	사기 패턴 8종(보이스피싱·몸캠피싱·자금세탁체인·대포통장 등) 룰 매칭 + 신뢰도 점수	pattern_library.py , pattern_analyzer.py
증거 분석	패턴별 필수/선택 증거 체크리스트 → 완성도·기소 준비도 평가	evidence_analyzer.py
네트워크 분석	연결도(degree) 허브 탐지, 공범망 추적(공유 계좌·전화·IP 순회), SNA 2-mode→1-mode 투영	graph_service.py , /api/v1/network/*
법률 RAG (v2)	조문·판례 하이브리드 검색(BM25+벡터+RRF+rerank) → 근거 인용 답변, 폐쇄망은 BM25-only 강등	legal_rag_service.py , /api/v1/legal/*
외부 파트너 API	키 인증 REST API 37종: text-to-cypher, 그래프 조회, 패턴 분석, ETL, 워크플로 등 (티어별 rate limit)	routes_api.py (/api/v1/*)
외부 read-only 조회	타 기관용 안전 조회 3종(read/dump/schema) — 쓰기 구문 차단, 별도 토큰	routes_graph_read.py

영역	기능	구현 위치
관리자	파트너 API 키 발급·관리 대시보드, 시스템/DB 모니터링	<code>routes_admin.py</code> , <code>monitoring_service.py</code>

5. 시스템 아키텍처

5.1 요청 흐름 (자연어 질의 한 건의 여정)

사용자 질문 "익명 피의자가 보유한 계좌 보여줘"

|

▼

[① 규칙 기반 사전 Router + LRU 캐시] `ai_service.py`

답답 (GENERAL) · 위험질의 (GUARD: 쓰기명령, 프롬프트 인젝션)를

LLM 호출 전에 정규식으로 차단 - 843ms 조기 응답

| 수사 질의만 통과 (의도분류: gpt-4o-mini, JSON 강제)

▼

[② LangGraph 에이전트 - 8노드 상태그래프] `langgraph_agent.py`

router → (PATH | QUERY 분기) → context_retrieval → schema_fetching

→ synthesis(Cypher 생성) → execution(실행)

→ 실패/0건 시 reflection → synthesis 재시도 (최대 2회)

└ 생성 LLM: 사내 vLLM sLLM(qwen25-t2c-v42) - 15s timeout

└ 폴백: GPT-4o 자동 전환 (총 16s 내)

└ 3중 검증: 스키마 화이트리스트 · 쓰기명령 차단 · 엣지 방향 자동교정(40+ 규칙)

└ few-shot 동적 주입(약점 카테고리별), Native Cypher-SQL wrapper 자동 변환

└ 전 질의 감사 로그(TB_AUDIT_LOG), 멀티턴 세션(엔티티 50개 컨텍스트)

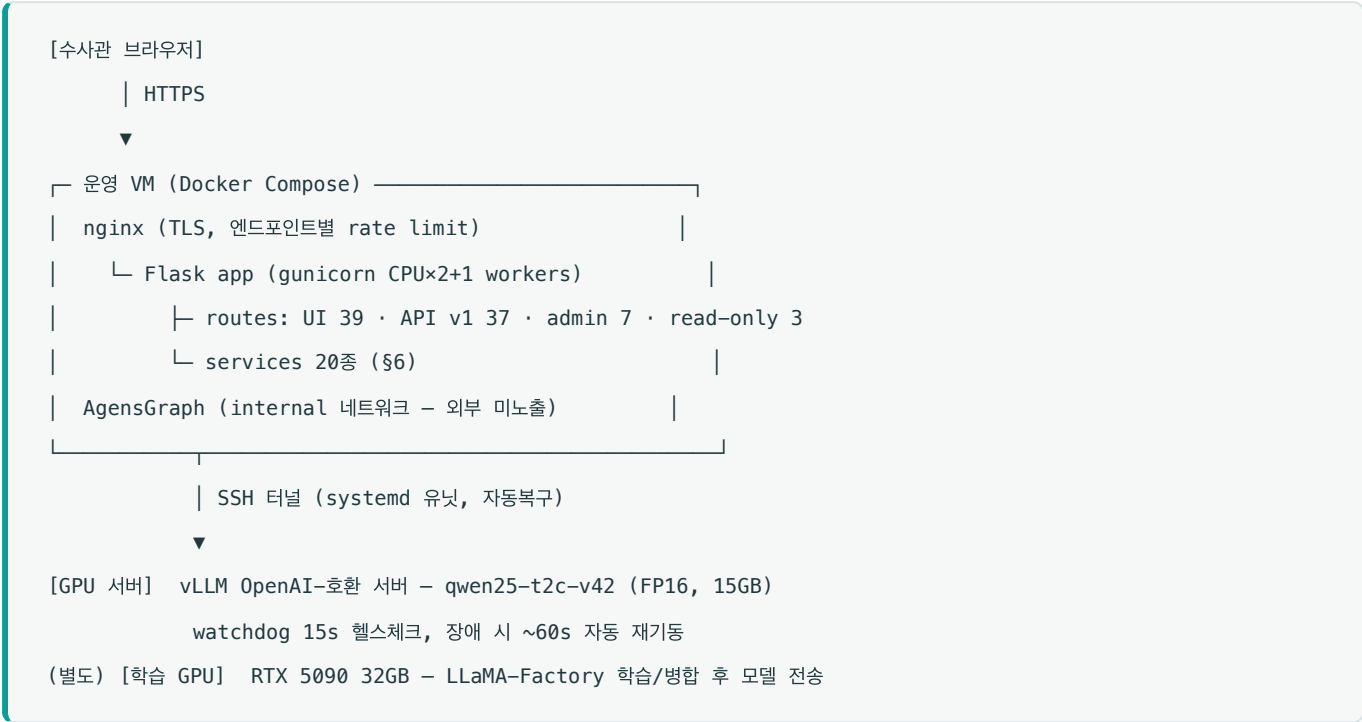
▼

[③ AgensGraph] `graph_path` 화이트리스트 → SET `graph_path` → Cypher 실행

▼

[④ Cytoscape.js 시각화] 노드 색 인코딩 · 확장/경로 UX · RDB 역추적 키(bridge_key) 표시

5.2 컴포넌트 배치 (운영 기준)



5.3 기술 스택

계층	기술	비고
프론트	Cytoscape.js + dagre, Vanilla JS, Pretendard/FontAwesome	전부 로컬 벤더링(CDN 미사용 — CSP·폐쇄망 대응)
백엔드	Flask 3.0 (Factory + Blueprint 4개), 서비스 레이어(classmethod 패턴)	Python 3.10
에이전트	LangGraph 0.6 (상태그래프), openai SDK 2.x	sLLM↔GPT-4o 풀백
그래프 DB	AgensGraph 2.13 (PostgreSQL 기반), psycopg2 커넥션 풀	RDB+GDB 단일 인스턴스
LLM 학습	LLaMA-Factory, QLoRA 4bit(nf4), LoRA r=64/α=128	RTX 5090 단일 GPU
LLM 서빙	vLLM (OpenAI-호환), systemd/watchdog 자가치유	A100 MIG·RTX 6000 Ada 검증
검색(RAG)	자체 BM25(한글 bigram) + 임베딩 + RRF + LLM rerank	신규 런타임 의존성 0
인프라	Docker Compose 3종, gunicorn+nginx, GitHub Actions CI	폐쇄망 USB 번들 체계

6. 코드 구조 (디렉토리 맵)

```
coop_v1.0/
├─ app/                                # Flask 애플리케이션 (~17,500 LOC)
│  └─ __init__.py                      # 앱 팩토리: blueprint 등록, 보안헤더(CSP/HSTS), gzip
│  └─ routes.py                        # UI·CRUD 39 엔드포인트 (질의/탐색/ETL/모델러/DB관리)
│  └─ routes_api.py                    # 파트너 API v1 37 엔드포인트 (전부 API 키 인증)
│  └─ routes_admin.py                  # 관리자 7 (로그인/대시보드/키 관리)
│  └─ routes_graph_read.py             # 외부 read-only 조회 3 (쓰기 구문 차단)
│  └─ database.py                      # 커넥션 풀 + graph_path 화이트리스트 검증
│  └─ middleware/
│     └─ api_auth.py                  # API 키 인증(SHA-256, timing-safe, rate limit)
│     └─ services/ontology_service.py # ★ 온톨로지 SSOT (2,226 LOC)
│  └─ services/                       # 서비스 레이어 20모듈 - 주요:
│     └─ langgraph_agent.py           # Text2Cypher 에이전트 (8노드, 1,282 LOC)
│     └─ ai_service.py                # 의도 라우터 + 옛지 방향 교정
│     └─ graph_service.py             # 검색/확장/경로/허브/공범망 (1,510 LOC)
│     └─ etl_service.py               # CSV→그래프 적재
│     └─ rdb_to_graph_service.py       # RDB 49테이블→POLE 변환 (2,148 LOC)
│     └─ pattern_*.py, evidence_analyzer.py # 사기패턴 8종·증거 분석
│     └─ legal_rag_service.py          # 법률 RAG v2 (hybrid+RRF+rerank)
│     └─ few_shot_router.py, schema_mapper.py, osint_v37_postprocess.py 등
│     └─ ontology_service.py           # (SSOT re-export shim)
│  └─ templates/                      # index.html(워크벤치 7,000 LOC)·modeler·admin
│  └─ static/                          # tokens.css(디자인 토큰 SoT), vendor(로컬 벤더링)
├─ train/                              # 학습 설정 yaml 9종·실행/병합/업로드 스크립트·v3.7 학습셋
├─ data/                              # SFT 데이터셋 50+파일·시드 생성/증강 스크립트 26종·legal/ 코퍼스
├─ scripts/                           # 배포·적재·평가 유틸 (~50) - eval_legal_rag.py, deploy.sh, 백업 등
├─ benchmark_t2c_v2.py                 # ★ 자체 벤치마크 하니스 (232문항 23카테고리)
├─ results/                           # 벤치마크 결과 JSON 아카이브 (버전별 18+)
├─ tests/                             # 테스트 25파일 (CI 오프라인 게이트 7파일 171개)
├─ docs/                              # 기술 문서 84건 (§15 지도 참조)
├─ deploy/                            # nginx/gunicorn 설정, 폐쇄망 템플릿, 배포 문서
├─ docker-compose{,.cslee,.airgap}.yml # 로컬/운영/폐쇄망 3종
└─ age/                              # (벤더링) Apache AGE 소스 - 참조용 제3자 코드
```

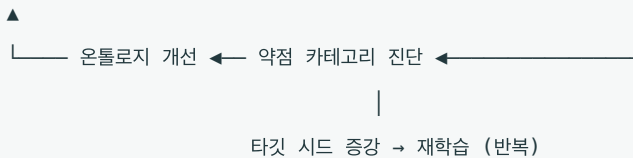
아키텍처 관례: routes는 얇게(검증·로깅), 로직은 `app/services/` 의 classmethod 서비스에. 코드 수정은 항상 `app/services/` (활성본)에 한다. 온톨로지만 예외적으로 `app/middleware/services/ontology_service.py` 가 SSOT이고 `app/services/ontology_service.py` 는 호환용 re-export다.

7. AI 파이프라인 — 어떻게 만들어졌나

이 프로젝트의 심장. "공개 벤치마크도, 학습 데이터도 없는 도메인에서 특화 모델을 만드는 전 과정"을 자체 구축했다.

7.1 전체 루프

온톨로지 정의(SSOT) → 학습 데이터 생성 → QLoRA 파인튜닝 → 벤치마크 평가



7.2 학습 데이터 — 31,226샘플을 증강 비용 \$0.11에

- **하이브리드 생성**: 온톨로지 템플릿 × 실 DB 값으로 규칙 시드 634개 생성 → GPT-4o-mini로 시드당 자연어 변형 8개(문체·동의어·어순, Cypher는 보존) → 5,700개 증강(비용 ~\$0.11) → 기존 데이터 25,526개를 SQL-wrapped→Native Cypher로 일괄 정제
- **분포 설계**: hop 수(0~4+, var-hop, shortestPath)별 비중을 의도적으로 배분, 13-stratum 층화 분할로 train 28,109 / eval 3,117 — **eval 셋은 버전 간 고정**(공정 비교)
- **포맷 교훈**: ShareGPT 포맷 + qwen 템플릿 조합에서 assistant 응답이 label 마스킹되어 "eval_loss 0.0002인데 아무것도 학습 안 됨" 버그 발견 → OpenAI messages 포맷 전환으로 해결
- **학습·서빙 프롬프트 완전 일치** 원칙: 온톨로지 스키마·금지 규칙을 시스템 프롬프트에 주입하며, 학습 때와 서빙 때 동일 프롬프트 사용

7.3 파인튜닝 — 단일 GPU 12회+ 사이클

RTX 5090(32GB) 한 장에서 QLoRA 4bit(nf4)로 7B 모델을 사이클당 4~7시간에 학습. 설정: LoRA r=64/α=128/dropout 0.05, target 7모듈(all-linear), lr 1e-4 cosine, 3 epochs, cutoff 1536.

버전	전략	결과(문항 수 기준)	판정
DeepSeek-Coder-7B	초기 검증	—	한국어 특화로 전환
EXAONE-3.5-7.8B ×4	10K~26K 샘플	깨진 토큰	❌ Blackwell GPU/transformers/custom code 호환성 → 표준 아키텍처로 교체 결정
Qwen v37_v1	ShareGPT 포맷	eval_loss 0.0002(가짜)	❌ label masking 버그
Qwen v37_v2	messages 전환	42.8% (152문항)	✅ 정상 학습 시작점
v38	약점 진단 시드 1,923	63.2% (+20.4p)	진단 기반 증강의 효율 입증
v39 → v40	보강 지속	74.3%(202) → 81.5% (232)	V4.0 대응

버전	전략	결과(문항 수 기준)	판정
v41	회귀 특화 830	79.3%	트레이드오프 확인, 폐기
v42 + Router	v40 어댑터 위 균형 970 + 규칙 라우터	86.6% (201/232)	🏆 운영 표준
v43	780 시드 단독 신규 학습	60.8% (-60문항)	✗ catastrophic forgetting — 부검 후 continue-learning 표준 수립
v44 (계획)	v42 어댑터 continue (lr 1/5, 1 epoch)	목표 89~91%	H2 로드맵

참고 기준선: GPT-4o zero-shot(스키마 인컨텍스트) 95.4%. 7B 로컬 모델로 이 격차를 추적·축소하는 구조다.

7.4 평가 — 벤치마크를 직접 만든다 (`benchmark_t2c_v2.py`)

- 142→152→202→**232문항**, **23카테고리**로 진화 (단일노드/1-hop 5종/체인/메타조건/v3.7 신규패턴/가드레일 등)
- **8종 자동 채점**(규칙 기반): 쿼리 구조 → 컬럼 정합 → 노드/엣지 화이트리스트 → 기대 엣지 적중률 → 신규 엣지 정확도 → 쓰기 금지 → 거절 응답 매칭
- **카테고리별 회귀 게이트(-5p 차단)**: "평균은 오르는데 특정 카테고리가 죽는" 문제를 시스템으로 방지. 전 결과는 `results/` 에 JSON 아카이브
- 핵심 교훈(문서화됨): *eval_loss가 낮다고 좋은 게 아니다 — 다양성 있는 holdout 벤치마크만이 진실*

7.5 시스템 보정 — 학습 없이 올린 정확도

- **규칙 기반 사전 Router**: 모델이 약한 잡담/위험질의 분기를 정규식+캐시로 처리 → **+5.6p (81.0→86.6%)**, 부수 효과로 DDL·프롬프트 인젝션 차단. 보고서에는 "Router 개선분 ≠ 모델 실력"을 분리 명시
- **엣지 방향 자동 교정**(POLE 규칙 40+), **few-shot 동적 주입**(약점 카테고리별 예시 top-k), 응답속도 스프린트(reflection 사전 차단 -3s, gzip -80.8%, 라벨 인덱스 18종으로 점 조회 10~100×)

7.6 서빙 — 제약 조건 엔지니어링

- **CUDA 12.2 고정 드라이버의 A100 MIG**에서 최신 vLLM이 동작 불가(torch 불일치) → `vllm 0.6.3.post1 + torch cu121 + transformers 4.46.3` **버전 핀 레시피** 확립 (`docs/VLLM_SETUP_GUIDE.md`)
- **자가치유**: vLLM watchdog(15s 헬스체크, 장애 ~60s 자동복구) + SSH 터널 systemd 유닛(부팅 자동기동). 운영 함정 3종 문서화(pkil self-match, orphan worker의 GPU 점유, 포트 미해제)
- 이기종 3환경(학습→서빙→운영 VM) 간 15GB 모델 전송·무결성 검증 파이프라인. 운영 체감: **자연어→그래프 e2e 1.4초**, sLLM 장애 시 GPT-4o 16초 내 자동 폴백

8. 온톨로지 — 도메인 데이터 모델

SSOT: `app/middleware/services/ontology_service.py` (`KICSCrimeDomainOntology`) + 명세 문서
`docs/CCOP_ONTOLGY_V4.0.md`

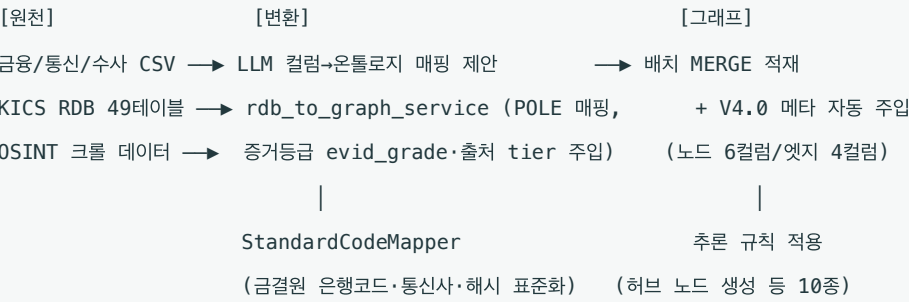
8.1 구조: POLE 정렬 6계층, 25노드 / 53엣지 / 추론규칙 10종

계층	노드 예시
Source	<code>vt_src</code> (데이터 출처)
Case	<code>vt_case</code> (사건), <code>vt_petition</code> (진정서), <code>pt_cluster</code> (진정서 군집 허브)
Person	<code>vt_psn</code> (인물, <code>is_anonymous</code> 성명불상 플래그), <code>vt_org</code> (조직)
Object	<code>vt_bacnt</code> (계좌), <code>vt_telno</code> (전화), <code>vt_ip</code> , <code>vt_site</code> , <code>site_cluster</code> (피싱 캠페인 허브), <code>vt_file</code> , <code>vt_id</code> , <code>vt_email</code> , <code>vt_crypto</code> , <code>vt_dev</code> (단말, <code>relay_station</code> 중계기), <code>vt_atm</code> , <code>vt_vhcl</code> , <code>vt_impersonation</code> (사칭)
Location	<code>vt_loc</code>
Event	<code>vt_transfer</code> (이체), <code>vt_call</code> (통화), <code>vt_msg</code> , <code>vt_access</code> , <code>vt_movement</code>

8.2 버전 진화 (설계가 어떻게 발전했나)

- **V2 (2월)**: Case→Actor→Action→Evidence 4-Layer 인지 모델, KICS 표준 정렬 (이 4계층 모델은 현재 CSV 자동매핑용으로 존속)
- **V3.3~3.6**: POLE 6계층 재편, 사칭을 엣지에서 노드로 승격(법 조문 반영), 23노드/52엣지
- **V3.7 (핵심 리모델링)**: 진정서 군집·피싱 캠페인을 표현하던 `clusters_with` $O(n^2)$ 엣지를 폐기하고 허브 노드 (`pt_cluster` / `site_cluster`)로 재설계 — 캠페인은 HTML **SimHash 지문** + **UnionFind**로 군집화. 불법중계기 추론 규칙 (IMEI 공유 전화 3대+) 등 수사 실무 패턴을 스키마화 → 25/53
- **V4.0 (통합 SSOT)**: 도메인 사용 매트릭스·노드 식별자 표준·시각화 표준(L5)·추론 규칙을 메타로 격상. 학습 데이터·ETL·시각화·문서가 전부 이 단일 정의를 참조
- 설계 근거 연구: RDF/OWL 대비 property graph 채택 분석, 상용 수사 도구(IBM i2) 비교 분석

9. 데이터 적재 파이프라인



- 대용량: GIN 인덱스 + 청크 배치 적재. 실적: `osint_ontology` 그래프 689만 노드/1,094만 엣지 적재·운영 (상시 적재 파이프라인은 H2 계획)
- RDB 역추적성: 그래프 노드마다 `bridge_key` (원천 테이블/PK)를 유지 — 시각화에서 원천 데이터로 되돌아갈 수 있다 (증거 무결성 관점)

10. 보안 설계

수사 데이터라는 특성상 **fail-closed**(설정 누락 시 잠김)가 원칙이다.

층	구현
입력 검증	<code>graph_path</code> 화이트리스트 정규식(<code>^[a-zA-Z_][a-zA-Z0-9_]*\$</code>) 전 경로 강제 — SQL injection 차단
쿼리 가드	생성 Cypher의 쓰기명령(DELETE/SET/MERGE/DROP...) 다층 차단: 라우터 GUARD → LangGraph 검증 → 외부 read-only API 자체 필터
프론트 인젝션	"이전 지시 잊어" 류 패턴 사전 차단 (Router)
인증	파트너 API 키 SHA-256 저장 + <code>hmac.compare_digest</code> (timing-safe) + fcntl 파일락, Bearer(기관별)와 X-API-Key(조회 토큰) 이원화
시크릿	<code>SECRET_KEY</code> / <code>ADMIN_PASSWORD</code> 미설정 시 기동 차단/기능 비활성 — 기본값 백도어 없음
전송·헤더	nginx TLS 강제(HTTP→301), HSTS/CSP/X-Frame-Options, 엔드포인트별 차등 rate limit (LLM 경로 10r/m 등)
네트워크	운영 compose는 DB를 <code>internal</code> 네트워크에 격리(외부 미노출), 클라우드 방화벽 소스 IP 화이트리스트
감사	전 자연어 질의 감사 로그(TB_AUDIT_LOG), 쿼리 로깅 커서
검증	위 항목들이 보안 테스트 27개로 회귀 고정 (CI 게이트)

11. 배포와 운영 (3개 환경)

환경	Compose	특징
로컬 개발	<code>docker-compose.yml</code>	소스 volume mount, <code>python run.py</code> (:5002) 또는 docker(:5001)
인터넷 운영 VM	<code>docker- compose.cslee.yml</code>	app+DB+nginx 3컨테이너, DB internal 격리, 헬스체크, 수동 배포 (<code>scripts/deploy.sh</code> , 자동 롤백)
폐쇄망(air- gap)	<code>docker- compose.airgap.yml</code>	사전 빌드 이미지 반입(app은 build 대신 image), sLLM은 <code>--profile gpu</code> 로 분리

폐쇄망 설치 체계 (`docs/AIRGAP_VISIT_INSTALL_RUNBOOK.md` , 374줄 현장 런북): - **1차/2차 위상 분리**: 리스크 낮은 인프라 (Docker+DB+앱+nginx, ~4-5GB 번들)를 먼저, 리스크 높은 GPU 스택(드라이버+vLLM+모델, ~30GB)을 나중에 — LLM 없이도 앱이 정상 기동하는 구조(lazy client) 덕분에 가능 - USB(exFAT) 반입 + `sha256sum -c` 무결성 검증, 대상 하드웨어 RTX 6000 Ada ×8(NVLink 부재 → tensor-parallel 미사용 판단), 1차↔2차 전환은 `.env` 세 줄 - Docker 이미지 다이어트: 죽은 의존성 제거 근거를 문서화하며 **7.8GB→~1GB**

운영 현황: 클라우드 VM에서 3컨테이너 healthy 가동, 그래프 9개(운영 기본 `tccop_graph_v6`), 사내 vLLM 서빙으로 외부 API 의존 없이 e2e 동작.

12. 품질 관리 (테스트·CI·벤치마크)

- **테스트**: 25파일, 테스트 함수 190여 개. 이 중 **CI 오프라인 게이트 171개**(DB/LLM/네트워크 불필요 — 순수 함수와 mock으로 설계): 보안 27 · 패턴/네트워크 27 · API 영속화 25 · 온톨로지 무결성 24 · T2C 헬퍼 20 · ETL 메타 14 · 법률 RAG 34
- **알려진 버그의 회귀 고정**: 실제 발견된 버그(같은 라벨 필수노드 붕괴)를 `xfail(strict)` 로 기록 — 고치면 테스트가 알려주는 구조
- **CI** (`.github/workflows/deploy.yml`): dev push/PR마다 ① 온톨로지 SSOT import 무결성 ② 앱 팩토리 기동 ③ 게이트 pytest. 배포(CD)는 의도적으로 수동(런북 기반)
- **모델 품질**: §7.4의 232문항 벤치마크 + 카테고리 회귀 게이트가 모델 릴리스의 게이트 역할. 결과 JSON은 `results/` 에 버전별 아카이브되어 보고서와 1:1 대응
- **문서 문화**: 설계 결정마다 근거 문서(84건) — 온톨로지 버전별 설계서, 모델 세대별 평가 보고서, 실패 부검(FINAL_REPORT §4), 운영 함정 기록

13. 개발 타임라인

시기	마일스톤
2026.01	플랫폼 v1: Flask+AgensGraph+Cytoscape.js, ETL, API 키 인증, 법률 RAG v1(ChromaDB — 이후 제거)
02	온톨로지 V2(4-Layer)·KICS 표준화·RDB 표준화, 멀티홉 정밀 확장, 초기 파인튜닝 실험(DeepSeek)
03-04	성능/부하 테스트, vLLM 서버 벤치마크, EXAONE 학습 시리즈, 벤치마크 v3.2 자동화
05	집중 연구기 : V3.7 온톨로지 리모델링 → 31,226샘플 구축 → EXAONE 실패 부검·Qwen 전환 → label masking 버그 발견 → v38/v39 → V4.0 SSOT → v40~v43 사이클 → catastrophic forgetting 부검
06	v42+Router 86.6% 운영 표준 확정 , 사내 vLLM 자체 서버 전환+자가치유, 운영 VM e2e(1.4s), 폐쇄망 1차 자산+런북, H2 연구계획
07	보안 하드닝(Phase 0-3)·디자인 토큰 SoT·분석 테스트+CI 게이트 (PR 워크플로 전환), 법률 RAG v2 재구축 (hybrid+RRF+rerank+평가 체계)

진행 중(H2 2026 계획): v44 continue-learning(목표 92%+), 온톨로지 SoT 단일화, OSINT 상시 적재(100만 건/h), agentic 분석 고도화, 폐쇄망 production 배포 — docs/H2_2026_RESEARCH_PLAN.md

14. 빠른 시작

```
# 1) 로컬 실행
pip install -r requirements.txt
cp .env.example .env          # DB_*, OPENAI_API_KEY (또는 SLLM_ENDPOINT) 설정
python run.py                  # http://localhost:5002

# 2) Docker
docker-compose up -d          # app:5001, agensgraph:5432

# 3) 테스트 (오프라인 게이트)
pytest -q tests/test_security.py tests/test_pattern_network_analysis.py tests/test_legal_rag.py # 등 7파일

# 4) 모델 벤치마크 (vLLM 서빙 중일 때)
python benchmark_t2c_v2.py --endpoint http://<vllm>:8000/v1 --mode t2c_v37

# 5) 법률 RAG 코퍼스 적재/평가
python scripts/ingest_legal_corpus.py --dry-run
python scripts/eval_legal_rag.py --mock-embeddings
```

환경변수: 필수 `DB_NAME/DB_USER/DB_PASSWORD/DB_HOST/DB_PORT`, LLM은 `OPENAI_API_KEY` 또는 `SLLM_ENDPOINT/SLLM_MODEL_NAME` (온프레미스), 선택 `EMBEDDING_ENDPOINT/EMBEDDING_MODEL_NAME/RAG_RERANK` (법률 RAG). 보안 필수값(`SECRET_KEY` / `ADMIN_PASSWORD`)은 미설정 시 fail-closed.

15. 더 읽을 문서 지도

주제	시작점	심화
온톨로지	CCOP_ONTOLOGY_V4.0.md (현행 명세)	ONTOLOGY_FINAL_ARCHITECTURE_v3.7.md (설계서), NODE_EDGE_REFERENCE.md , SEMANTIC_INFERENCE_RULES.md , RDF_OWL_ANALYSIS.md
Text2Cypher 모델	T2C_V40_V41_V42_FINAL_REPORT_20260529.md (최종 비교+실패 부검)	TEXT2CYPHER_V37_EVAL_REPORT.md , T2C_SFT_DEVELOPMENT_PLAN.md , MODEL_TRAINING_GUIDE.md
서빙/GPU	VLLM_SETUP_GUIDE.md	CHECKPOINT_20260616.md (운영 전환 기록, 자가치유 검증)
배포·운영	VM_DEPLOY_OPERATIONS_GUIDE.md	AIRGAP_VISIT_INSTALL_RUNBOOK.md , AIRGAP_DEPLOY_GUIDE.md , DEV_WORKFLOW.md
외부 연동 API	API_GUIDE.md , EXTERNAL_CYPHER_QUERY_HOWTO.md	EXTERNAL_GRAPH_QUERY_GUIDE.md , PARTNER_ONBOARDING.md , PARTNER_DATA_STANDARD.md
데이터/RDB	DATABASE_ARCHITECTURE.md , V40_RDB_TO_GRAPH_MAPPING.md	RDB_STANDARDIZATION_v3.6.md , OSINT_INGESTION_PIPELINE_DESIGN.md
법률 RAG	LEGAL_RAG_V2_DESIGN.md	—
로드맵	H2_2026_RESEARCH_PLAN.md	IMPROVEMENT_PLAN.md , 05_OPEN_ISSUES.md
경쟁/ 포지셔닝	I2_COMPARISON_ANALYSIS.md (IBM i2 대비)	—

16. 용어집

용어	뜻
KICS	형사사법정보시스템 — 한국 형사사법 표준 데이터 체계. 온톨로지가 이 표준의 용어·코드에 정렬됨
POLE	Person·Object·Location·Event — 수사 데이터 모델링의 국제 관행. 본 프로젝트는 +Source·Case 6계층
Cypher	그래프 DB 질의 언어 (<code>MATCH (a)-[r]->(b) RETURN ...</code>). AgensGraph는 이를 SQL로 감싸 실행
vt_ / pt_	그래프 노드 라벨 접두어 (예: <code>vt_bacnt</code> 계좌, <code>vt_psn</code> 인물, <code>pt_cluster</code> 진정서 군집 허브)
Text2Cypher (T2C)	자연어→Cypher 자동 변환 과제. 이 프로젝트의 AI 핵심
sLLM	온프레미스에서 도는 소형 특화 LLM. 본 프로젝트는 Qwen2.5-7B 파인튜닝 (<code>qwen2.5-t2c-v42</code>)
SFT / LoRA / QLoRA	지도 파인튜닝 / 저랭크 어댑터 학습(원본 동결) / 4bit 양자화+LoRA — 32GB GPU에서 7B 학습을 가능케 한 조합
Router	LLM 호출 전 규칙(정규식)으로 잡담·위험질을 차단하는 사전 분기기 — 정확도 +5.6p, 가드레일
Reflection	쿼리 실행 실패 시 오류를 피드백으로 재생성하는 자가수정 루프 (LangGraph)
RRF	Reciprocal Rank Fusion — BM25와 벡터 검색의 순위를 무투닝으로 융합하는 방법 (법률 RAG)
SSOT	Single Source of Truth — 온톨로지 정의가 코드 한 곳에 있고 전 시스템이 참조하는 원칙
Air-gap (폐쇄망)	인터넷과 물리적으로 분리된 망. USB 반입 설치·외부 API 0 동작이 요구사항
bridge_key	그래프 노드에 새겨 둔 원천 RDB 테이블/PK — 시각화에서 원본 데이터로 역추적하는 키
catastrophic forgetting	파인튜닝 시 새 지식이 기존 능력을 덮어쓰는 현상. v43 실패의 원인이자 continue-learning 표준의 배경

이 문서는 리포 실측 조사(2026-07) 기준이며, 수치가 코드와 어긋나면 코드가 우선한다. 갱신 시 §1 표와 §13 타임라인을 함께 갱신할 것.